Automated Detection of Pork Rasher Packaging Defects with YOLO

1. Engineering problem and scope

This project builds and evaluates a computer-vision method that finds packaging and product-placement defects in trays of pork rashers and decides whether each tray passes or fails inspection.

Context and problem. Food-packaging lines rely on manual inspection to catch defective products before distribution, and automated inspection aims to reduce that reliance (Monash University, 2026). This is a manufacturing-automation problem: a camera-based quality-control step judged on accuracy, cost per image and speed relative to the line. Given one RGB image of a packaged tray, the method must detect and localise each defect (class label and bounding box) and return a tray-level PASS or FAIL. The five classes are `loose-meat`, `packaging-error`, `twisted-meat`, `unsealed` and `wrinkle`.

Intended use. A screening step that flags trays for rejection or human review. A missed defective tray ships and a false alarm wastes product and inspector time, so the main tray-level measure is recall (the share of defective trays rejected), reported with the false-alarm rate and 95% confidence intervals. No line speed is given, so 30 frames per second is an assumed real-time target, and no recall target was fixed in advance.

Scope. The project compares three YOLO configurations on accuracy and speed, evaluates the tray verdict and where it fails, and tests real-time suitability. Out of scope are defect severity, fat percentage, counting rashers, segmentation and integration into a real line.

2. Inputs, outputs and data

The system takes one tray image and returns defect detections plus a PASS/FAIL verdict. It was built on a public Roboflow data set of 3,549 images.

Inputs and outputs.

Input: one RGB tray image (JPG or PNG; 720 × 540 pixels here, resized internally to 640 or 480 px) and a confidence threshold in (0, 1], 0.35 here.

Output: a list of detections (class, confidence from 0 to 1, box `[x1, y1, x2, y2]` in pixels) and a verdict, **FAIL** if any detection has confidence at or above the threshold, otherwise **PASS**. A missing file raises `FileNotFoundError`. An unsupported extension, an unreadable image or a threshold outside (0, 1] raises `ValueError`. Tests cover these cases (Section 5.1), and each function's inputs and outputs are in the HTML documentation in `docs/`.

Data: *Pork Rasher Error (Packaging)* (2025), licensed CC BY 4.0: about 1,955 originals expanded to 3,549 images by augmentation in the Version 4 export (Monash University, 2026), with three augmented copies per original in train and one copy in valid or test in the frames inspected.

Split	Images	Defect-free images	loose-meat	packaging-error	twisted-meat	unsealed	wrinkle
train	3,349	684	66	1,712	72	1,055	18
valid	120	14	3	34	3	81	0
test	80	12	3	23	4	49	0

Data reflects the original download. The classes are heavily imbalanced: `wrinkle` has no valid or test boxes and cannot be evaluated, and `loose-meat` and `twisted-meat` have only 3 or 4 each, so their scores are indicative only. Defect-free images stay in training as background.

Data leakage found and fixed. Roboflow names augmented copies `<original>.rf.<hash>.jpg`, so the text before `.rf.` identifies the original frame. Twelve frames appeared in train and also in valid (8) or test (4), and one in both valid and test, so the model would have been tested on images it had effectively seen. `src/clean_splits.py` deleted every such train copy and removed the shared valid/test frame from test: 36 training images and 1 test image. The splits now hold 3,313 train, 120 valid and 79 test images (68 defective and 11 defect-free test trays) with no shared original.

3. Computational approach

The method fine-tunes a pretrained YOLOv8 detector on the tray images and applies a confidence-threshold rule to give a PASS/FAIL verdict. YOLO was chosen because it outputs the boxes and confidences needed for localisation and a tray threshold, is fast enough to test real-time use directly, and suits about 3,300 training images through transfer learning (alternatives: Section 4).

How it works. YOLOv8 (Jocher et al., 2023) is a single-stage detector, a design introduced by Redmon et al. (2016): one forward pass predicts every box, class and confidence. A convolutional backbone extracts features at several scales, a head predicts boxes and class scores, and non-maximum suppression (NMS) merges overlapping predictions. The weights start from a model pretrained on COCO (Lin et al., 2014), so only the new five-class head and later layers must adapt (transfer learning).

Tray rule and settings. A tray fails if at least one detection has confidence of 0.35 or more. The threshold was chosen on validation data only, by maximising recall minus false-alarm rate (ties to the lower threshold), and fixed before the test split was examined. The three runs share the settings below and differ only in model and input size.

Setting	Value
Starting weights	COCO-pretrained `yolov8n.pt` or `yolov8s.pt`; 5 classes
Training	At most 50 epochs, early stopping after 15; batch size 16; AdamW with automatic learning rate; seed 0
Model selection within a run	Best epoch by validation fitness (equals mAP50-95 in this release)

Setting	Value
Augmentation	Ultralytics defaults: mosaic (off for the last 10 epochs), HSV shifts, horizontal flip, scale ±50%, translation ±10%, random erasing
Precision and caching	Automatic mixed precision; images cached to disk
Hardware and software	Colab Tesla T4; Python 3.13; PyTorch 2.11; Ultralytics 8.4.184 (run 1), 8.4.185 (runs 2 and 3)

Assumptions. Labels are correct, defects are visible at 640 or 480 px, test images resemble training images, no original frame is shared between splits, and one confident detection condemns a tray wherever it lies (tested in Section 5.3).

Implementation. A Colab notebook calls three documented, tested modules in `src/`: `clean_splits.py`, `tray_inspector.py` and `evaluation.py` (box overlap, tray metrics, threshold choice, label statistics). Each configuration was trained once with one seed.

4. Alternatives and model selection

Three YOLO configurations were compared, and YOLOv8n at 640 px (`v8n_640`) was selected on validation mAP50-95 by a rule written down before the test split was examined.

Run	Model and input	What it tests	Epochs run	Training Time (min)	Val mAP50	Val mAP50-95
`v8n_640`	YOLOv8n (3.1 M parameters), 640 px	Baseline	50	37	0.610	**0.377**
`v8s_640`	YOLOv8s (11.1 M parameters), 640 px	Does a larger model help?	47 (best at 32)	38	0.578	0.336
`v8n_480`	YOLOv8n, 480 px	Does a smaller input hurt?	42 (best at 27)	21	0.566	0.364

A trivial tray-level baseline, "always FAIL", is correct for 68 of 79 test trays (86%), which shows why raw accuracy misleads (Section 5.3).

Selection rule (fixed in advance). Compare candidates on validation results only and select the highest validation mAP50-95. Use the test split once, and report all three runs whichever scores highest on the test. Headline metrics are mAP50, mAP50-95 and mAP50 over the two well-populated classes (`packaging-error`, `unsealed`).

Decision. `v8n_640` has the highest validation mAP50-95 (0.377, against 0.364 and 0.336) and about 3.3 times fewer operations than `v8s_640` (Section 6). The evidence is weak: the gaps are 0.01 to 0.04 on 121 validation boxes from one seed each, `loose-meat` and `twisted-meat` have 3 boxes each, validation also chose the epochs and the tray threshold, `v8n_640` may still have been improving at epoch 50, and all candidates are YOLO variants (a different family is future work).

5. Validation and results

On the 79-tray test set the selected model rejects 85% of defective trays and falsely rejects 1 of 11 good ones, but it passes 10 defective trays, and the test set is too small to separate the three candidate models.

5.1 Verification: is the code correct?

Sixty-four pytest tests pass (Python 3.13, pytest 8.4.2; report in `tests/report.html`). They cover leakage removal (including that re-running it removes nothing further), the PASS/FAIL rule and its boundary, the Wilson interval, invalid inputs, and the evaluation helpers behind the reported numbers, checked against hand-computed values. A fake model stands in for YOLO, so they verify the logic around the detector, not its accuracy.

5.2 Detection accuracy on the test set

The test split has 79 images and 79 boxes. mAP50 is the mean over classes of average precision at an IoU (overlap) of 0.5; mAP50-95 averages IoU thresholds from 0.5 to 0.95 and so also rewards tight boxes.

Run	mAP50	mAP50-95	mAP50, `packaging-error` + `unsealed`	Precision	Recall
`v8n_640` (selected)	0.534	0.234	0.616	0.473	0.511
`v8s_640`	0.567	0.268	0.634	0.747	0.464
`v8n_480`	0.544	0.229	0.709	0.473	0.617

Per-class mAP50 on the test set (test boxes in brackets):

Run	`loose-meat` (3)	`packaging-error` (23)	`twisted-meat` (4)	`unsealed` (49)
`v8n_640`	0.863	0.421	0.041	0.811
`v8s_640`	0.753	0.440	0.245	0.829
`v8n_480`	0.665	0.565	0.092	0.853

- **No run is clearly best.** `v8s_640` leads on mAP50 and mAP50-95 and `v8n_480` on the two well-populated classes, while `v8n_640` is last or second on each. Gaps of 0.02 to 0.04 are within what one or two boxes can move.
- **Validation was optimistic.** For `v8n_640`, mAP50 fell from 0.610 to 0.534 and mAP50-95 from 0.377 to 0.234, suggesting loosely placed boxes as well as misses.
- **Rare-class scores are noise,** and `packaging-error`, the most common training class, scores only 0.42 to 0.57 in every run.

5.3 Tray-level PASS/FAIL

A tray is defective if its label file has any box. The threshold was chosen on validation (14 defect-free trays), where recall minus false-alarm rate peaked at 0.772 at 0.35; the peak is flat (0.10 scored 0.767),

so the threshold is not sharply determined. Test result for `v8n_640` at 0.35, with 95% Wilson intervals (Wilson, 1927):

Measure	Result	95% CI
Defective trays rejected (any detection ≥ 0.35)	58 of 68 = 0.853	0.750 to 0.918
Rejected with a detection on a real defect (IoU ≥ 0.3)	54 of 68 = 0.794	0.684 to 0.873
Rejected with a detection on a real defect (IoU ≥ 0.5)	52 of 68 = 0.765	0.651 to 0.850
Good trays wrongly rejected	1 of 11 = 0.091	0.016 to 0.377

- **Accuracy is misleading.** The model is right on 68 of 79 trays (0.861), the same as "always FAIL", because 86% of test trays are defective. Balanced accuracy is fairer: 0.881 against 0.500.
- **The rule ignores location, so it flatters recall.** Six of the 58 rejections had no detection on a labelled defect at IoU 0.5, and four at IoU 0.3, so recall for correctly placed detections is 0.77 to 0.79.
- **Ten defective trays would ship,** and a PASS is right only about half the time (10 of 20 passed trays are good), so the model suits screening for human review, not replacing inspection. With 1 false alarm in 11 good trays, the false-alarm rate is barely measured (interval to 38%).
- **Single defects are caught less often.** All 10 misses carry one defect: 49 of 59 single-defect trays were rejected (0.83, CI 0.72 to 0.91) against 9 of 9 with two or more (CI 0.70 to 1.00), but the intervals overlap, so no difference is established.

5.4 Failure analysis

Each of the ten missed trays carries one labelled box (3 `packaging-error`, 3 `unsealed`, 3 `twisted-meat`, 1 `loose-meat`). By visual inspection, the missed `twisted-meat` defects are barely noticeable and produced no detection, the single false alarm lies on glare, and for several misses low-confidence detections lie near the labelled box but not on it. An edge-of-frame explanation was tested and rejected, because nearly every box lies near an image edge (within 10% of a side):

Boxes	Near an image edge	Share
Train, all boxes	2,140 of 2,887	0.74
Test, all boxes	72 of 79	0.91
Test, boxes in the 14 failure trays	14 of 16	0.88

5.5 What this validation cannot establish

- **Small samples and one seed:** 79 test trays, only 11 good, one seed per run, so differences of a few points are not significant and the false-alarm and rare-class results are indicative only.
- **One source:** performance on other lines, cameras or lighting is unknown.
- **Selection effects and labels:** validation chose the epochs and the threshold, so validation scores are optimistic (test was used once); label quality was not audited; the tests check logic, not detection accuracy.

6. Optimisation and performance

All three models run far faster than real time on a T4 GPU (77 to 126 FPS against a 30 FPS benchmark), so speed does not constrain the choice.

Method. One Colab session, the same 79 test images in memory, 30 warm-up predictions then three timed passes, GPU synchronised around every call. Times cover resizing, the forward pass and NMS, not disk reading or camera capture.

Run	Input (px)	Parameters (M)	GFLOPs	File size (MB)	Mean ms	Median ms	95th percentile ms	FPS
`v8n_640`	640	3.14	8.75	6.50	8.64	8.46	10.18	116
`v8s_640`	640	11.14	28.65	22.51	13.04	13.05	13.85	77
`v8n_480`	480	3.14	4.92	6.48	7.95	7.81	9.27	126

Real-time suitability. The 30 FPS target gives a 33 ms budget per frame. `v8n_640` averages 8.6 ms (116 FPS, about 3.9 times the throughput needed) and even `v8s_640` takes 13.9 ms at the 95th percentile. The line speed is not given, so 30 FPS is assumed, and a weaker plant PC would have less margin.

Trade-offs observed.

- **FLOPs overstate the cost of a bigger model:** `v8s_640` needs 3.3 times the operations of `v8n_640` but only 1.5 times the time.
- **A smaller input saves little time:** 480 px cuts FLOPs by 44% but mean time by 8%. Fixed costs (preprocessing, NMS, kernel launches) probably dominate, but this was not profiled, so it is a hypothesis.
- **The faster options cost little accuracy on tests**, within noise (Section 5).

Final design. `v8n_640` is kept: every candidate clears the real-time target by a wide margin, so trading accuracy for speed has no benefit, and `v8s_640` costs more computation and memory without a demonstrated gain. `v8n_480` is a fallback for weaker hardware. Model size, input size, the tray threshold and the overlap for a correctly placed detection (IoU 0.5 and 0.3) were varied as the sensitivity analysis.

Missing tests. No profiler trace (so where the time goes is inferred), no arithmetic-intensity estimate, no exported or half-precision inference, and no timing on edge hardware or a CPU.

7. Limitations and lessons learned

The results show that YOLOv8 can act as a tray screening aid, but small test data and a single seed per run mean they cannot rank the models or promise performance on another line.

Limitations. See Section 5.5. In addition, nothing was run on line hardware or a live camera, and the Ultralytics AGPL-3.0 licence would need review before commercial use.

Lessons learned

1. **Augmented data sets can leak across splits.** Thirteen frames crossed split boundaries; checking the original frame behind each file name caught it before training.
2. **Accuracy hides imbalance, and lenient rules hide errors.** "Always FAIL" matched the model's accuracy (0.861), and counting any detection as a catch gave 0.853 recall against 0.77 to 0.79 with correct placement.
3. **Test your own explanations.** The edge-of-frame explanation looked convincing in 14 images and failed once all boxes were counted.
4. **Fix rules before seeing test data, and make long jobs resumable.** Written selection and threshold rules kept the comparison honest, and self-resuming runs meant Colab resets cost minutes.

Future work (not tested). Add glare examples and more `twisted-meat`, `loose-meat` and `wrinkle` images, build a larger independent test set and repeat training over several seeds, compare a second detector family, make the tray rule location-aware.

8. AI use and reflection

Claude (Anthropic), used through the claude.ai chat interface (Claude Sonnet 5.5, medium setting), supported this project as a planning, coding, debugging and writing assistant, including the first draft of this report.

What it was used for. Scoping, Colab, GitHub and git setup, code for every pipeline stage (exploration, leakage cleaning, training, evaluation, tray inspector, tests, README), debugging, explaining concepts and results, and the first draft of this report from numbers I produced.

Approximate level of contribution. Claude wrote almost all of the code and outlined the structure of this report from results I ran and pasted into the chat. I made the final decision at each step using the project brief and the AI's suggestions, but the AI proposed several design choices that I accepted: the narrowed scope, the three-run comparison, the selection and threshold rules, and the leakage fix.

Why AI was used. I am new to object detection and was short on time. I had used Colab and GitHub in this unit before but needed my memory refreshed, and the AI gave me working code and setup instructions (including for YOLO training, testing and documentation) so I could run the whole workflow in the time available.

How AI output was checked

- Every number comes from the output I ran and pasted back from the notebook, and I re-checked each figure against the notebook outputs. I also questioned the AI's claims: asking whether the documentation met the brief exposed missing function-level documentation and tests, and asking what references it had used showed they were written from memory and needed checking.
- The 64 tests ran first in the AI's environment and then in Colab (Python 3.13.16, pytest 8.4.2), where all passed.
- I looked at the failure images myself. Where the AI offered an explanation, a test was run, and one was rejected (Section 5.4).

Errors made by the AI and how each was caught.

What the AI got wrong	How it was caught	Effect
Assumed red defect outlines on the images and suggested red-pixel detection	The project slide and brief showed bounding-box annotations	Switched to object detection
Guessed the dataset zip held three copies of the data	A listing showed 7,103 files, one clean export	Guess dropped
Led model weights to be committed to the repo	The commit output listed `.pt` files	Commit reset, `.gitignore` extended
Left a one-frame valid/test overlap unaddressed	The output showed `valid∩test: 1`	Frame removed from test before the comparison runs
Proposed that failures cluster at the image edge	Edge counts: 0.91 of test boxes against 0.88 in failure trays	Rejected and reported as a negative result
Gave a clone cell that was unsafe to re-run (it created a nested copy of the repo)	The output showed a repo path nested inside the repo	Folder deleted; the notebook skips the clone if the repo exists

Other errors and omissions: documentation was incomplete at first (two modules, and notebook functions without docstrings, until I asked whether it met the brief); a commit cell printed a warning about a stray zip file but committed it anyway; the early 28-minute estimate for the baseline run was too low (it took about 37); and a README draft contained two unchecked claims (a shared GPU, and a red overlay on the images), which were removed.

Decisions that remained mine. I chose Google Colab, although the AI's explanation leaned towards training locally on my own GPU. I selected the pork rasher packaging project (Dataset 3) from the unit's provided projects, decided which analyses to run and report, and accepted the AI's proposed three-run comparison and narrowed scope after weighing them against the project brief. I examined the failure images myself and decided what they show: the false alarm on glare, the faint `twisted-meat` misses and the near-miss detections.

Effect on my understanding and workflow. The AI let me complete a full object-detection workflow in the time available and helped with setup and debugging. It also made errors that I had to catch, so checking its work became part of my workflow. I understand most of the overall approach and results but not the exact code the AI wrote, which is a limitation of my understanding. Before the presentation I will go through the code line by line so that I can explain it without AI.

9. References

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8.0.0) [Computer software]. Ultralytics. https://github.com/ultralytics/ultralytics

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In D. Fleet, T. Pajdla, B. Schiele, & T. Tuytelaars (Eds.), *Computer vision – ECCV 2014* (Lecture Notes in Computer Science, Vol. 8693, pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

Monash University. (2026). *Project information – Dataset 3: Automated detection of pork rasher packaging errors and meat quality* [Unit project information]. MMA3001 Numerical Methods and Machine Learning, Monash University.

@Hello-2aqe0. (2025) *Pork rasher error (packaging)* [Data set]. Roboflow Universe. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779–788). IEEE. https://doi.org/10.1109/CVPR.2016.91

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association, 22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

Source code, tests, generated documentation and results tables are in the project's GitHub repository: https://github.com/opat0009-wq/pork-rasher-inspection